

BAZAR.COM

Lab 1 Report

Multi-tier Online Book Store

MICROSERVICES	REST API	SQLITE	DOCKER
Frontend + backend services	HTTP JSON endpoints	Persistent storage	Containerized deployment

Course	Distributed Operating Systems
Student Name	Ameer AboShams & Zeina Anabtawi
Student ID	12217464 & 12217834
Instructor	Dr. Samer Arandi
Date	24/5

01

Project Overview

Purpose, scope, and Lab 1 requirements.

This report presents Lab 1 of the Bazar.com project. The system is a small online bookstore implemented using a multi-tier microservices design. The frontend receives client requests and forwards them to backend services. The catalog service manages book information, while the order service processes purchase operations and records orders.

Lab 1 Goal

The main goal is to build a lightweight distributed web application using REST APIs, persistent storage, and separate deployable services. The system should support search, info, and purchase operations and run the services as independent components.

FRONTEND	CATALOG	ORDER
Client entry point Receives search, info, and purchase requests.	Book data Stores title, topic, quantity, and price.	Purchases Checks stock, decreases quantity, and saves orders.
REST HTTP JSON Services communicate using REST endpoints.	SQLITE Persistence Database stores books and order records.	DOCKER Deployment Three containers run the application services.

02

System Architecture

Container layout and request flow.

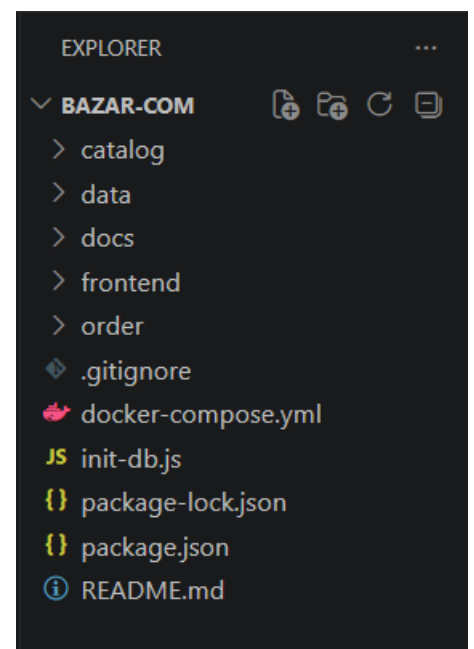
The system follows a two-tier web design.

The frontend is the single entry point for users.

The backend consists of two separate services: catalog and order.

The catalog service is responsible for read and update operations on book data.

The order service is responsible for purchase operations and communicates with the catalog service to verify availability before buying.



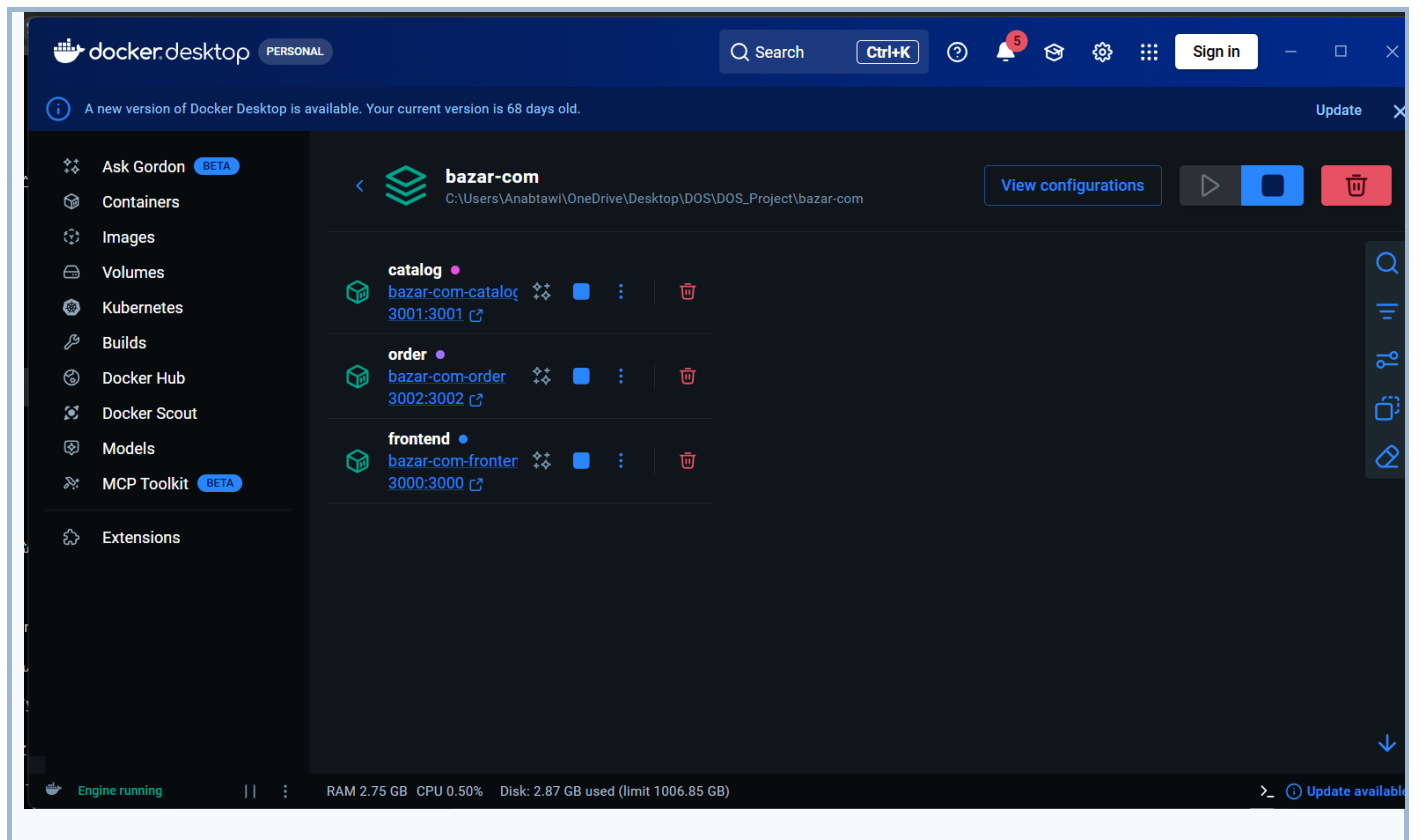


Figure 1. Docker Desktop or terminal output showing the three running containers.

Component	Container	Port	Responsibility
Frontend	bazar_frontend	3000	Client entry point; forwards search, info, and purchase requests.
Catalog	bazar_catalog	3001	Handles catalog queries and stock/price updates.
Order	bazar_order	3002	Processes purchases and saves order records.
Database	bazar.db	SQLite file	Stores books and orders persistently on disk.

Request flow

Search and info requests go from the client to the frontend and then to the catalog service. Purchase requests go from the client to the frontend, then to the order service. The order service queries the catalog, updates stock, and writes the order to the database.

03

Docker Deployment

docker-compose.yml and service startup.

Each service has its own Dockerfile and runs as a separate container. Docker Compose starts the three containers together and maps each internal service port to the host machine.

docker-compose.yml excerpt - three microservices

```
services:
  catalog:
    build: ./catalog
    container_name: bazar_catalog
    ports:
      - "3001:3001"
    environment:
      - DB_PATH=/app/data/bazar.db
    volumes:
      - ./data:/app/data

  order:
    build: ./order
    container_name: bazar_order
    ports:
      - "3002:3002"
    depends_on:
      - catalog
    environment:
      - DB_PATH=/app/data/bazar.db
      - CATALOG_URL=http://catalog:3001
    volumes:
      - ./data:/app/data

  frontend:
    build: ./frontend
    container_name: bazar_frontend
    ports:
      - "3000:3000"
    depends_on:
      - catalog
      - order
    environment:
      - CATALOG_URL=http://catalog:3001
      - ORDER_URL=http://order:3002
```

```

#28 [order] resolving provenance for metadata file
#28 DONE 0.0s
[+] up 6/6
✓ Image bazar-com-catalog Built
✓ Image bazar-com-order Built
✓ Image bazar-com-frontend Built
✓ Container bazar_catalog Recreated
✓ Container bazar_order Recreated
✓ Container bazar_frontend Recreated
Attaching to bazar_catalog, bazar_frontend, bazar_order
bazar_catalog | [Catalog] Server running on port 3001
bazar_catalog | [Catalog] Connected to SQLite database
bazar_order | [Order] Server running on port 3002
bazar_order | [Order] Connected to SQLite database
bazar_frontend | [Frontend] Server running on port 3000

View in Docker Desktop View Config Enable Watch Detach

```

```

PS C:\Users\Anabtawi\OneDrive\Desktop\DOS\DOS_Project\bazar-com> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
5416aa9ec7f2   bazar-com-frontend   "docker-entrypoint.s..." 42 seconds ago Up 2 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
9e478cb988bf   bazar-com-order      "docker-entrypoint.s..." 42 seconds ago Up 2 minutes   0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
a424157c9f64   bazar-com-catalog    "docker-entrypoint.s..." 43 seconds ago Up 2 minutes   0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
PS C:\Users\Anabtawi\OneDrive\Desktop\DOS\DOS_Project\bazar-com>

```

Figure 2. docker compose up --build output showing frontend, catalog, and order services running.

04 Implementation Details

Major services and how each one works.

Feature	Implementation	How it is verified
Search by topic	Frontend forwards /search/:topic to catalog /query/topic/:topic.	Postman returns matching book IDs and titles.
Book details	Frontend forwards /info/:id to catalog /query/item/:id.	Response includes title, topic, quantity, and price.
Purchase	Frontend forwards /purchase/:id to order service.	Stock decreases and order is saved.
Persistence	SQLite database is mounted using Docker volume.	Database viewer shows books and orders tables.
Error handling	Services return 404/400/500 messages when needed.	Invalid item IDs return Item not found.

The implementation also stores the ordered quantity in the orders table. The default quantity is one copy, which matches the basic Lab 1 purchase behavior. This makes the order log more descriptive without changing the main requirement.

04 Frontend Service

Client entry point and request forwarding.

The frontend service is the only service directly accessed by the client. It receives requests, performs basic routing, and forwards each request to the correct backend service. It does not directly update the database.

Endpoint	Method	Forwarded To	Purpose
/search/:topic	GET	Catalog /query/topic/:topic	Search books by topic.
/info/:id	GET	Catalog /query/item/:id	Return details for one book.
/purchase/:id	POST	Order /purchase/:id	Buy a book item.

frontend/server.js excerpt - search forwarding

```
const CATALOG_URL = process.env.CATALOG_URL || 'http://localhost:3001';
const ORDER_URL = process.env.ORDER_URL || 'http://localhost:3002';

app.get('/search/:topic', async (req, res) => {
  const topic = req.params.topic;
  const response = await axios.get(
    `${CATALOG_URL}/query/topic/${encodeURIComponent(topic)}`
  );
  res.json({
    message: 'Search completed successfully',
    items: response.data.items
  });
});
```

frontend/server.js excerpt - info and purchase forwarding

```
app.get('/info/:id', async (req, res) => {
  const id = req.params.id;
  const response = await axios.get(`${CATALOG_URL}/query/item/${id}`);
  res.json({
    message: 'Item details retrieved successfully',
    item: response.data.item
  });
});

app.post('/purchase/:id', async (req, res) => {
  const id = req.params.id;
  const response = await axios.post(`${ORDER_URL}/purchase/${id}`, req.body);
  res.json(response.data);
});
```

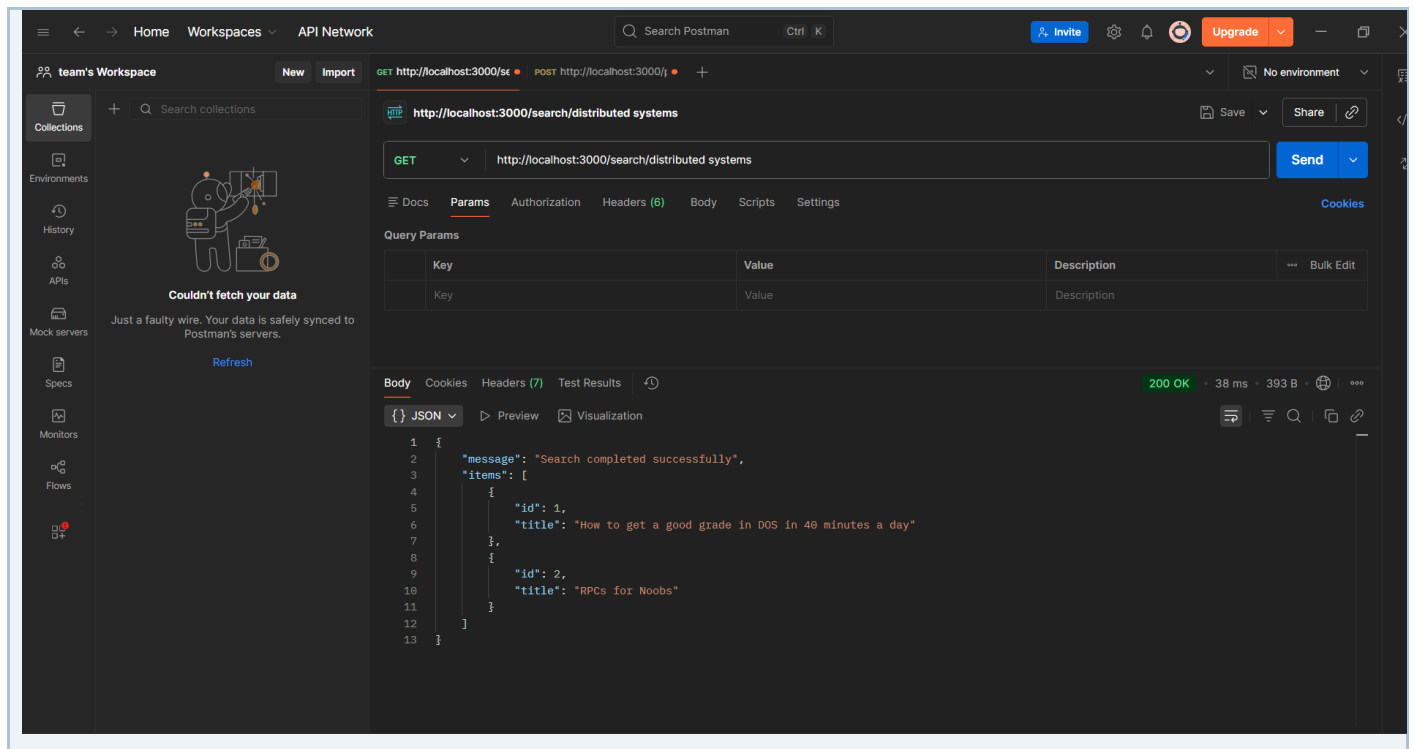


Figure 3. Frontend search request output from Postman.

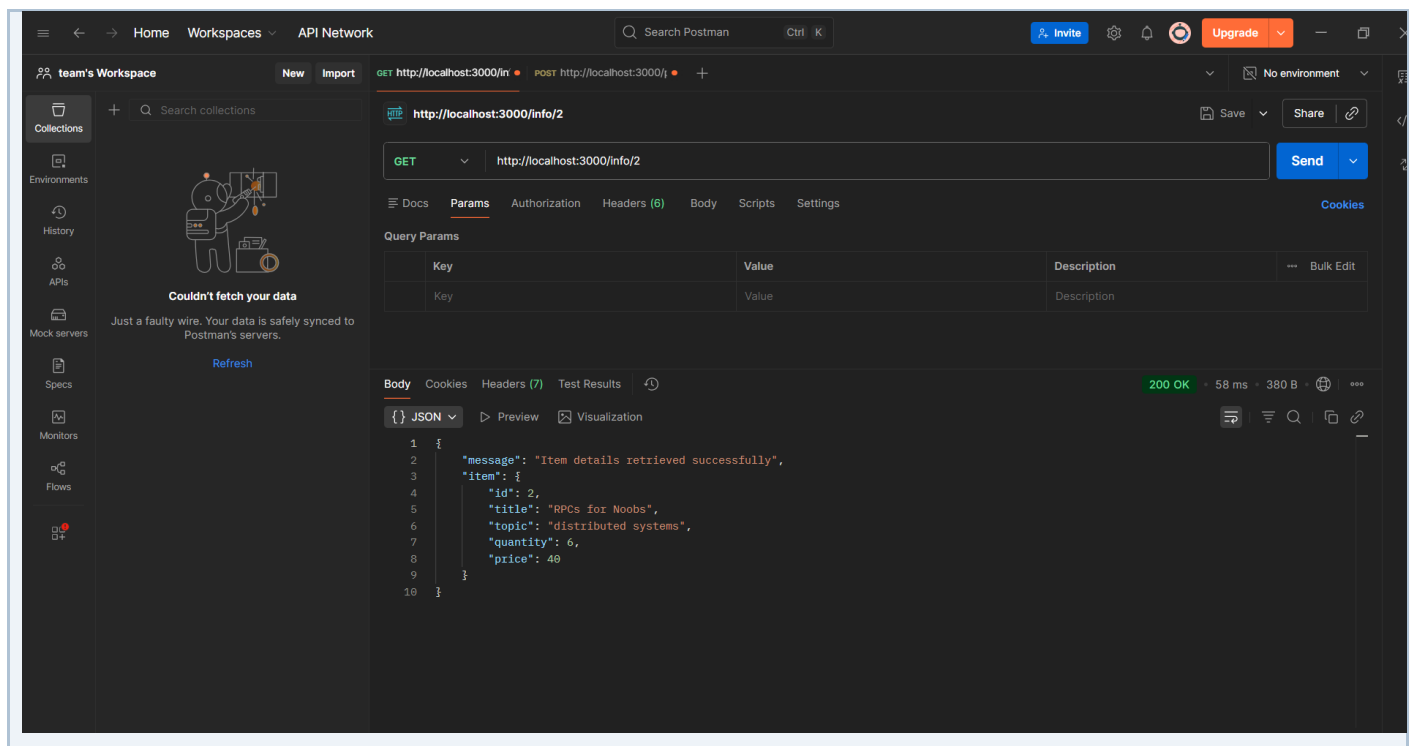


Figure 4. Frontend info request output from Postman.

frontend/server.js excerpt - frontend API routes

```
app.get('/search/:topic', async (req, res) => {
  const topic = req.params.topic;
  const response = await axios.get(`${CATALOG_URL}/query/topic/${encodeURIComponent(topic)}`);
  res.json({ message: 'Search completed successfully', items: response.data.items });
});

app.get('/info/:id', async (req, res) => {
  const id = req.params.id;
  const response = await axios.get(`${CATALOG_URL}/query/item/${id}`);
  res.json({ message: 'Item details retrieved successfully', item: response.data.item });
});

app.post('/purchase/:id', async (req, res) => {
  const id = req.params.id;
  const response = await axios.post(`${ORDER_URL}/purchase/${id}`, req.body);
  res.json(response.data);
});
```

05 Catalog Service

Query and update operations.

The catalog service maintains the book catalog. It supports query-by-topic, query-by-item, update stock, and update price operations. The frontend uses the query operations for search and info requests, while the order service uses query and stock update during purchases.

Endpoint	Method	Purpose
/query/topic/:topic	GET	Returns all books that belong to a topic.
/query/item/:id	GET	Returns full details for one book item.
/update/stock/:id	POST	Increases or decreases the stock quantity.
/update/price/:id	POST	Updates the price of a specific book.

catalog/server.js excerpt - query item

```
// Query books by topic
app.get('/query/topic/:topic', (req, res) => {
  const topic = req.params.topic.toLowerCase();

  db.all(
    `SELECT id, title FROM books WHERE LOWER(topic) = ?`,
    [topic],
    (err, rows) => {
      if (err) {
        return res.status(500).json({
          message: 'Failed to retrieve books by topic'
        });
      }

      res.json({
        message: 'Books retrieved successfully',
        items: rows
      });
    }
  );
});
```



```

    }
  );
});

// Query one item by id
app.get('/query/item/:id', (req, res) => {
  const id = req.params.id;

  db.get(
    `SELECT id, title, topic, quantity, price FROM books WHERE id = ?`,
    [id],
    (err, row) => {
      if (err) {
        return res.status(500).json({
          message: 'Failed to retrieve item details'
        });
      }

      if (!row) {
        return res.status(404).json({
          message: 'Item not found'
        });
      }

      res.json({
        message: 'Item details retrieved successfully',
        item: row
      });
    }
  );
});

```

```

// Update stock quantity
app.post('/update/stock/:id', (req, res) => {
  const id = req.params.id;
  const change = Number(req.body?.change);

  db.get(`SELECT * FROM books WHERE id = ?`, [id], (err, book) => {
    if (!book) {
      return res.status(404).json({
        message: 'Item not found'
      });
    }

    const newQuantity = book.quantity + change;

    if (newQuantity < 0) {
      return res.status(400).json({
        message: 'Not enough stock available'
      });
    }

    db.run(
      `UPDATE books SET quantity = ? WHERE id = ?`,
      [newQuantity, id],
      () => {
        res.json({
          message: 'Stock updated successfully',
          item: {
            id: book.id,
            title: book.title,
            quantity: newQuantity
          }
        });
      }
    );
  });
});

```

```

    });
  }
);
});
});

// Update book price
app.post('/update/price/:id', (req, res) => {
  const id = req.params.id;
  const price = Number(req.body?.price);

  db.run(
    `UPDATE books SET price = ? WHERE id = ?`,
    [price, id],
    function () {
      if (this.changes === 0) {
        return res.status(404).json({
          message: 'Item not found'
        });
      }

      res.json({
        message: 'Price updated successfully',
        item: {
          id: Number(id),
          price
        }
      });
    }
  );
});
});

```

06 Order Service

Purchase operation and stock update.

The order service receives purchase requests. It first asks the catalog service for the requested item. If the item exists and enough stock is available, it calls the catalog stock update endpoint to decrease the quantity and then inserts a new record into the orders table.

Step	Action	Result
1	Receive POST /purchase/:id	The item ID is extracted from the URL.
2	Query catalog service	The order service checks item details and stock.
3	Validate availability	If stock is zero or insufficient, the request fails.
4	Update catalog stock	The quantity is decreased.
5	Insert order record	The order is saved in SQLite.

order/server.js excerpt - purchase logic

```
app.post('/purchase/:id', async (req, res) => {
  const id = req.params.id;
  const quantity = Number(req.body?.quantity ?? 1);

  const catalogResponse = await axios.get(`${CATALOG_URL}/query/item/${id}`);
  const book = catalogResponse.data.item;

  if (book.quantity < quantity) {
    return res.status(400).json({ message: 'Not enough stock available' });
  }

  const updateResponse = await axios.post(`${CATALOG_URL}/update/stock/${id}`, {
    change: -quantity
  });

  db.run(
    `INSERT INTO orders (item_id, title, quantity, status, ordered_at)
    VALUES (?, ?, ?, ?, ?)`,
    [book.id, book.title, quantity, 'SUCCESS', new Date().toISOString()]
  );
});
```

07 Database Design

SQLite tables and initial data.

The database is stored in data/bazar.db. It contains two main tables: books and orders. The books table stores the catalog items, while the orders table stores successful purchase records.

Table	Columns	Purpose
books	id, title, topic, quantity, price	Stores catalog data and current stock levels.
orders	id, item_id, title, quantity, status, ordered_at	Stores purchase history.
ID	Title	Topic
1	How to get a good grade in DOS in 40 minutes a day	distributed systems
2	RPCs for Noobs	distributed systems
3	Xen and the Art of Surviving Undergraduate School	undergraduate school
4	Cooking for the Impatient Undergrad	undergraduate school

```
CREATE TABLE IF NOT EXISTS books (
  id INTEGER PRIMARY KEY,
  title TEXT NOT NULL,
  topic TEXT NOT NULL,
  quantity INTEGER NOT NULL,
  price REAL NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS orders (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  item_id INTEGER NOT NULL,
  title TEXT NOT NULL,
  quantity INTEGER NOT NULL,
  status TEXT NOT NULL,
  ordered_at TEXT NOT NULL
);
```

id	title	topic	quantity	price
1	How to get a good grade in DOS in 40 minutes a day	distributed systems	10	50
2	RPCs for Noobs	distributed systems	6	40
3	Xen and the Art of Surviving Undergraduate School	undergraduate school	9	35
4	Cooking for the Impatient Undergrad	undergraduate school	11	35

Figure 5. SQLite viewer showing the books table.

id	item_id	title	status	ordered_at
1	1	How to get a good grade in DOS in 40 minutes a day	SUCCESS	2026-04-03T12:01:37.361Z
2	2	How to get a good grade in DOS in 40 minutes a day	SUCCESS	2026-04-03T13:10:04.484Z
3	3	RPCs for Noobs	SUCCESS	2026-04-03T13:11:20.731Z
4	4	Cooking for the Impatient Undergrad	SUCCESS	2026-04-03T13:18:28.599Z

Figure 6. SQLite viewer showing the orders table after purchase.

08 Testing Procedure and Output

Postman requests and expected results.

The following test cases verify the main functionality of the system. Each screenshot should show the request URL, HTTP method, status code, and returned JSON response.

Step	Request	Expected Result
1	GET /search/distributed systems	Returns books 1 and 2.
2	GET /search/undergraduate school	Returns books 3 and 4.
3	GET /info/2	Returns RPCs for Noobs with quantity and price.
4	POST /purchase/2	Returns purchase success and saves an order.
5	GET /info/2 after purchase	Quantity decreases by one.
7	POST /update/stock/2	Stock quantity changes successfully.
8	POST /update/price/2	Price changes successfully.

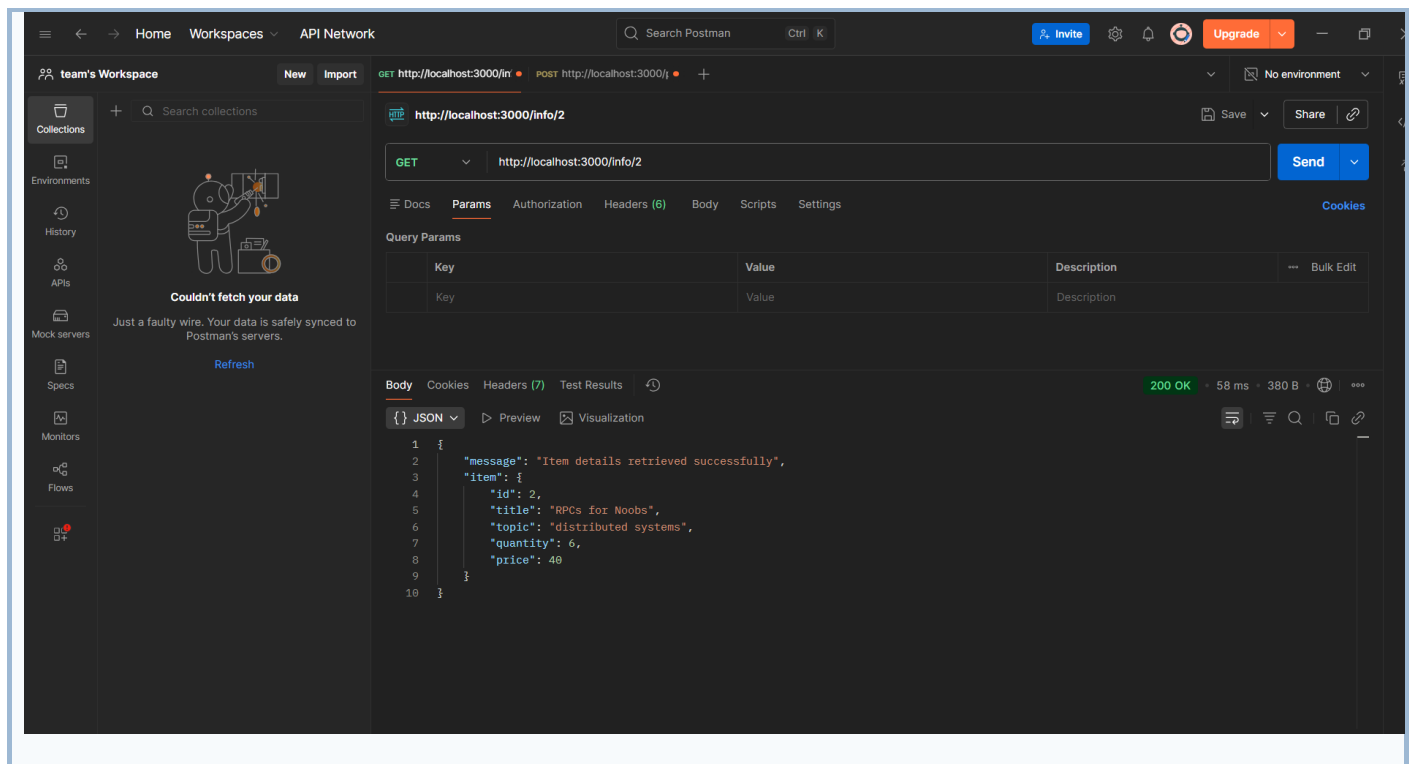


Figure 7. GET /info/2 response before purchase.

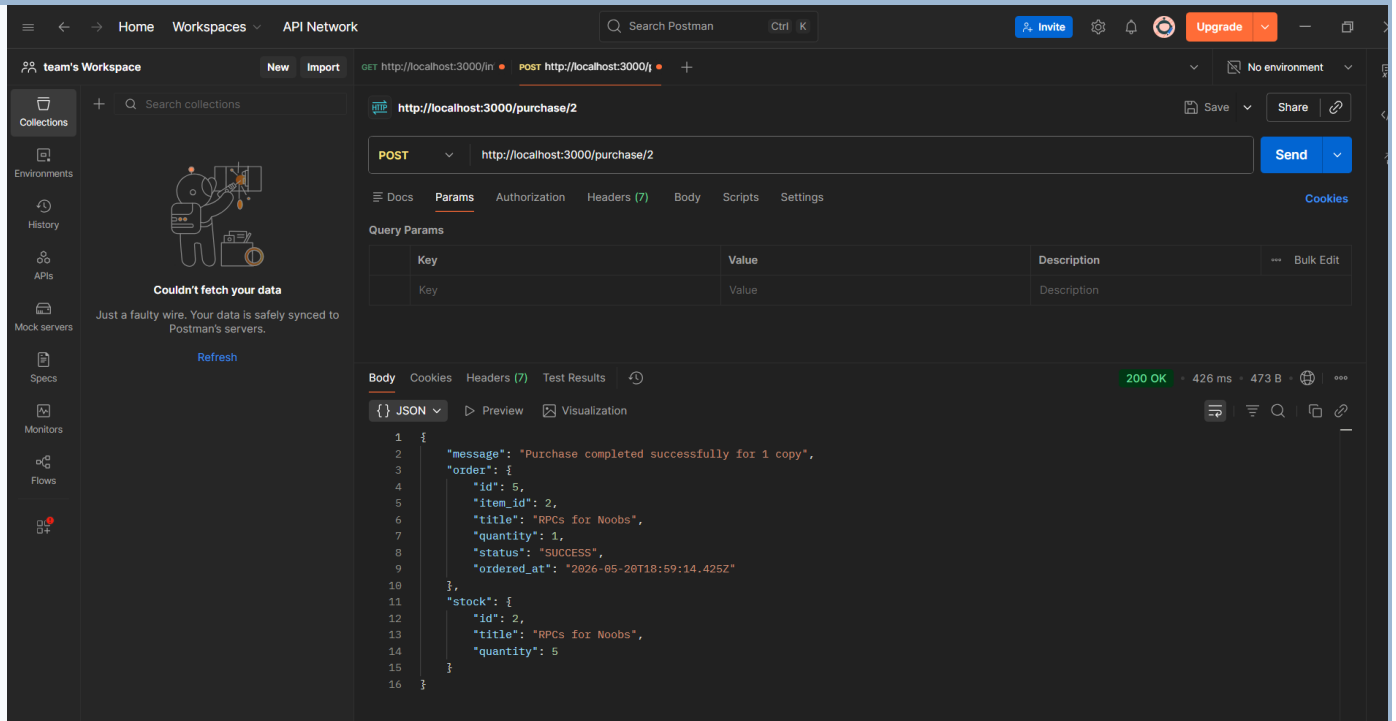


Figure 8. POST /purchase/2 response showing successful purchase.

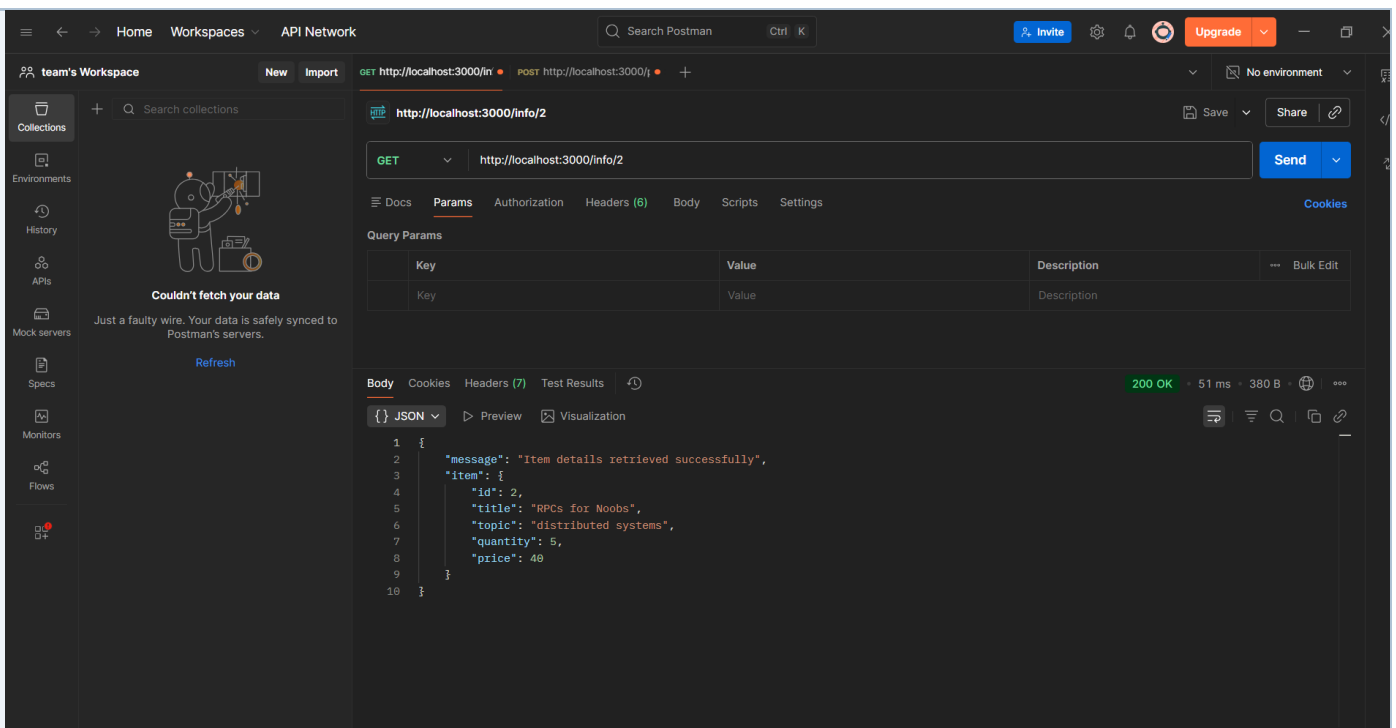


Figure 9. Item details after purchase showing updated stock.

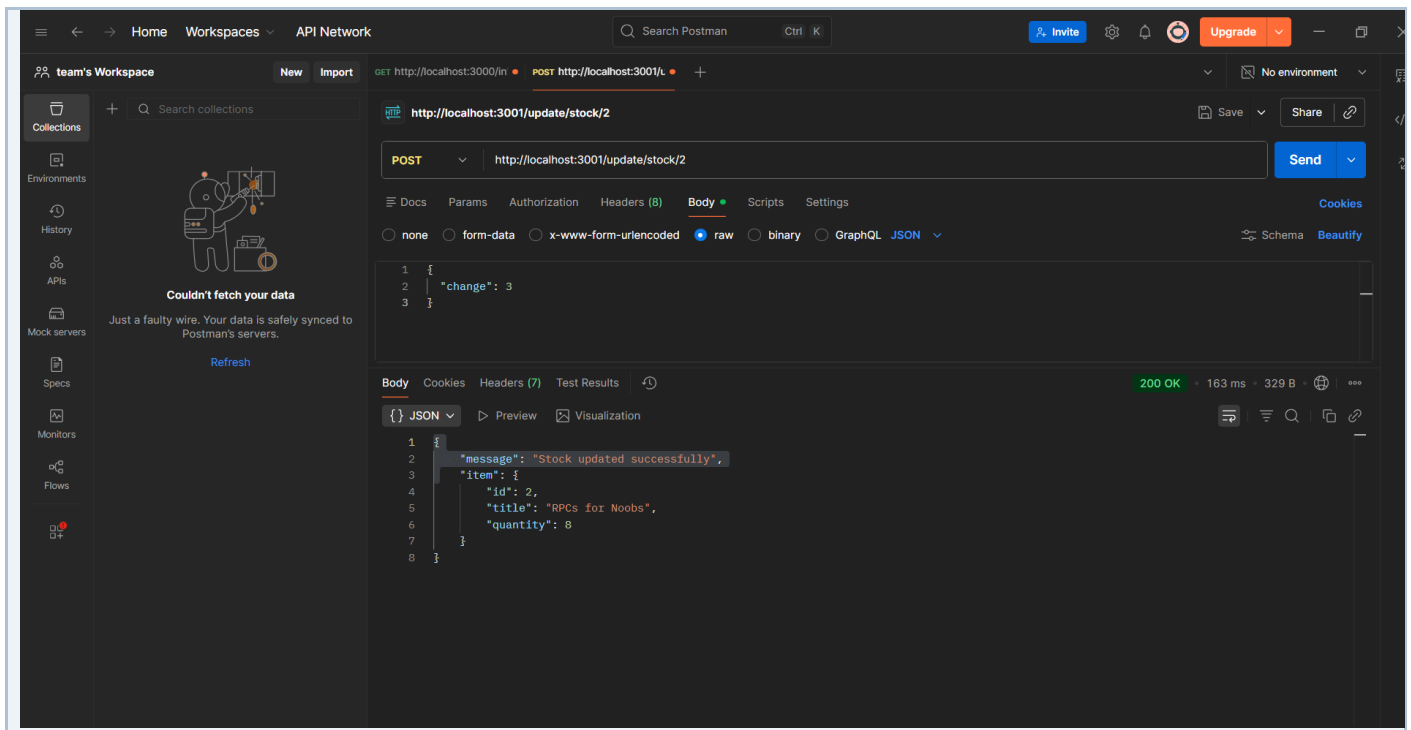


Figure 10. Catalog stock update output.

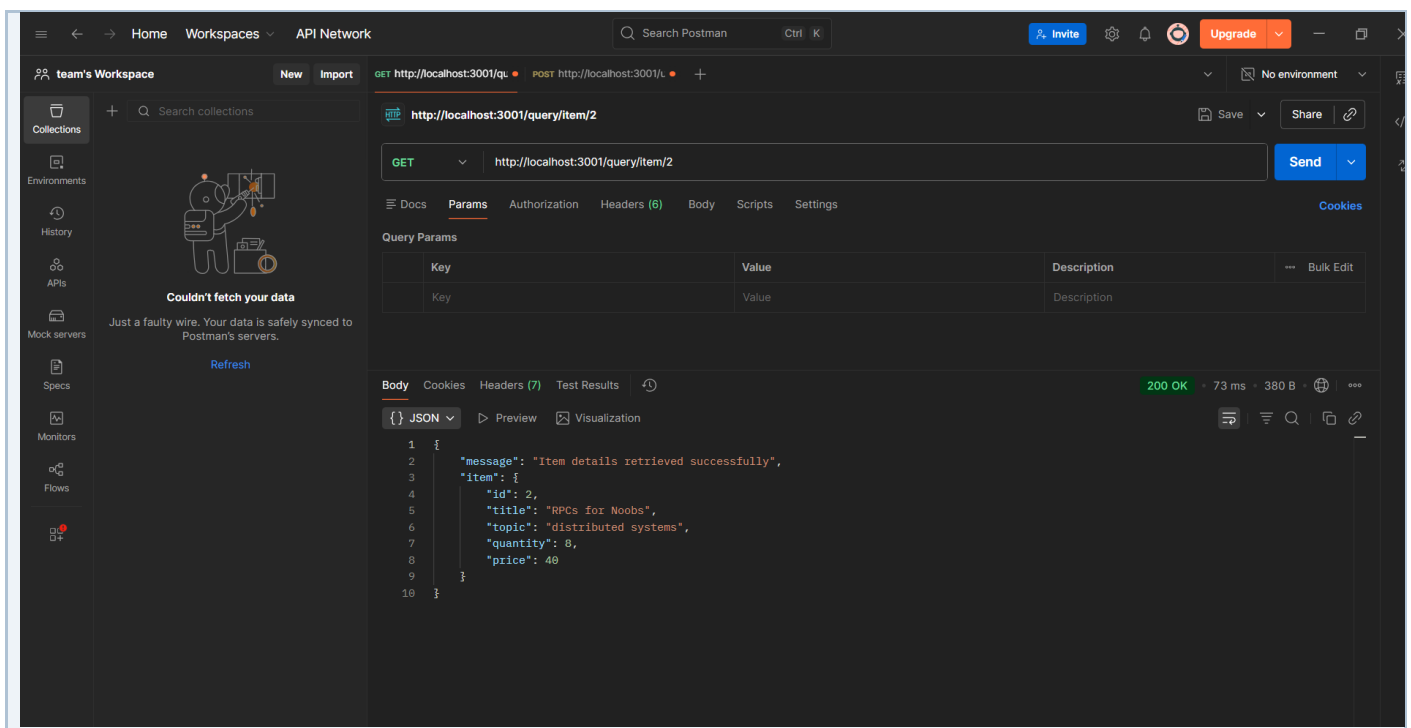


Figure 11. Item details after update showing updated stock.

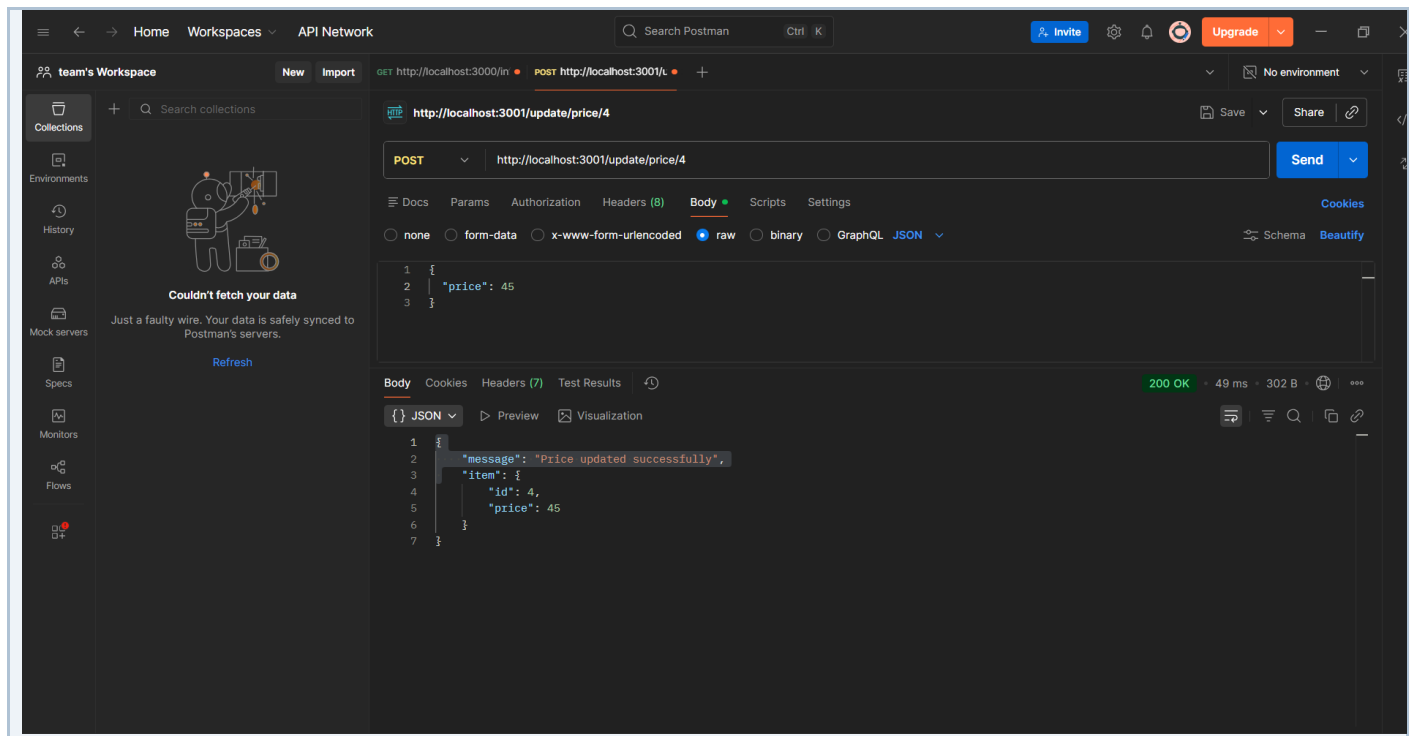


Figure 12. Catalog price update output.

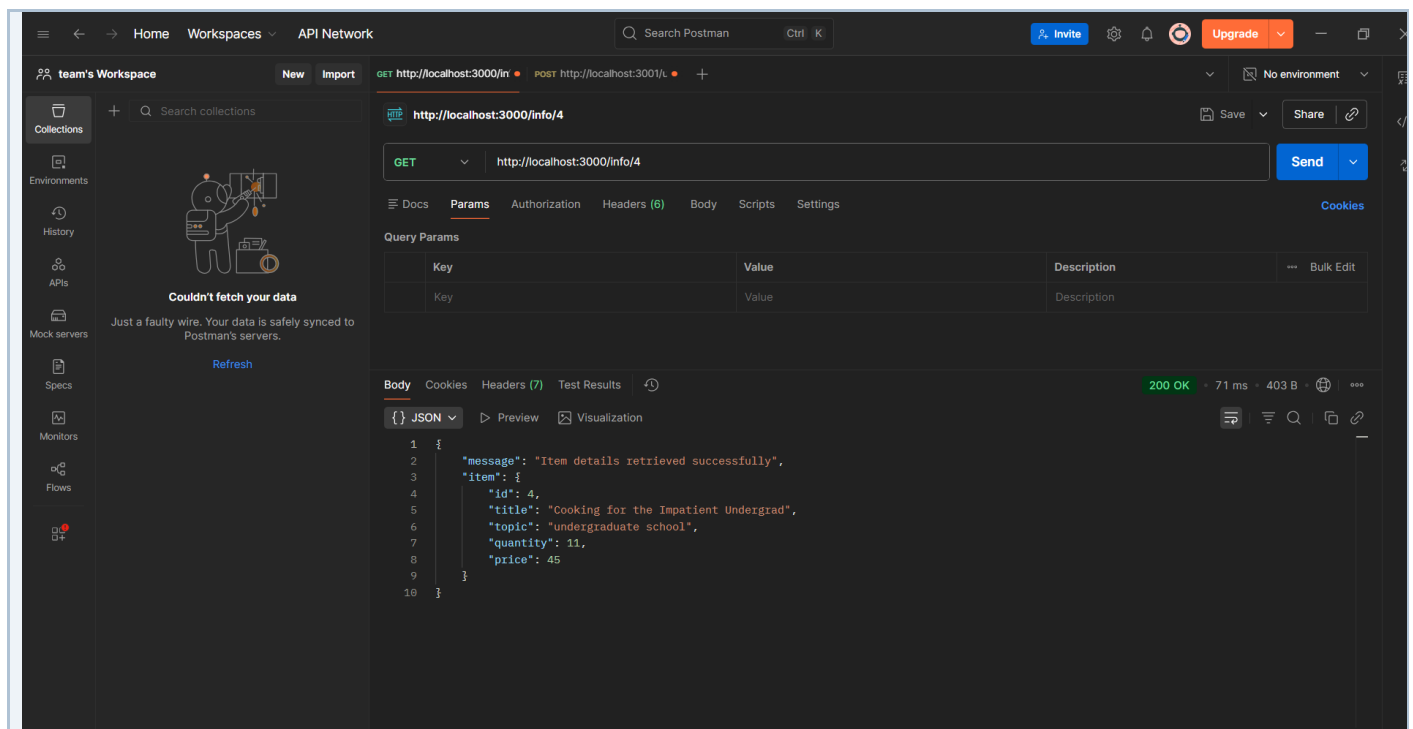


Figure 13. Item details after update showing updated price.

data > bazar.db

Rows: 5

Filter 5 rows...

Upgrade to PRO

	id	item_id	title	status	ordered_at	quantity
1	1	1	How to get a good grade in DOS in 40 minutes a day	SUCCESS	2026-04-03T12:01:37.361Z	
2	2	1	How to get a good grade in DOS in 40 minutes a day	SUCCESS	2026-04-03T13:10:04.484Z	
3	3	2	RPCs for Noobs	SUCCESS	2026-04-03T13:11:20.731Z	
4	4	4	Cooking for the Impatient Undergrad	SUCCESS	2026-04-03T13:18:28.599Z	
5	5	2	RPCs for Noobs	SUCCESS	2026-05-20T18:59:14.425Z	
+ 6						

Figure 14. Orders table after all.

data > bazar.db

Rows: 4

Filter 4 rows...

Upgrade to PRO

	id	title	topic	quantity	price
1	1	How to get a good grade in DOS in 40 minutes a day	distributed systems	10	50
2	2	RPCs for Noobs	distributed systems	8	40
3	3	Xen and the Art of Surviving Undergraduate School	undergraduate school	9	35
4	4	Cooking for the Impatient Undergrad	undergraduate school	11	45
+ 5					

Figure 15. Books table after all.

docker desktop PERSONAL

Search

Ctrl+K

Sign in

A new version of Docker Desktop is available. Your current version is 68 days old.

Update

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Docker Hub

Docker Scout

Models

MCP Toolkit BETA

Extensions

bazar-com

C:\Users\Anabta\OneDrive\Desktop\DOS\DOS_Project\bazar-com

View configurations

catalog	[Catalog] Server running on port 3001
catalog	[Catalog] Connected to SQLite database
order	[Order] Server running on port 3002
order	[Order] Connected to SQLite database
frontend	[Frontend] Server running on port 3000
frontend	[Frontend] Search request received for topic: distributed systems
catalog	[Catalog] Query by topic received: distributed systems
frontend	[Frontend] Search request received for topic: distributed systems
catalog	[Catalog] Query by topic received: distributed systems
frontend	[Frontend] Info request received for item: 2
catalog	[Catalog] Query by item received: 2
frontend	[Frontend] Purchase request received for item: 2
order	[Order] Purchase request received for item 2, quantity: 1
catalog	[Catalog] Query by item received: 2
order	[Order] Item 2 is available. Proceeding with stock update
catalog	[Catalog] Stock update request received for item 2, change: -1
catalog	[Catalog] Stock updated successfully for item 2. New quantity: 5
order	[Order] Order saved successfully for item 2

Figure 16. Docker logs.

09

Error Handling

Invalid requests and failure cases.

The system handles invalid cases by returning clear JSON messages. This makes the services easier to test and prevents silent failures.

Case	Request	Expected Response
Invalid item info	GET /info/99	404 Item not found.
Invalid purchase	POST /purchase/99	404 Item not found.
Not enough stock	POST /purchase/:id when quantity is unavailable	400 Not enough stock available.
Invalid stock update	POST /update/stock/:id with invalid change	400 Invalid stock change value.
Invalid price update	POST /update/price/:id with negative price	400 Invalid price value.

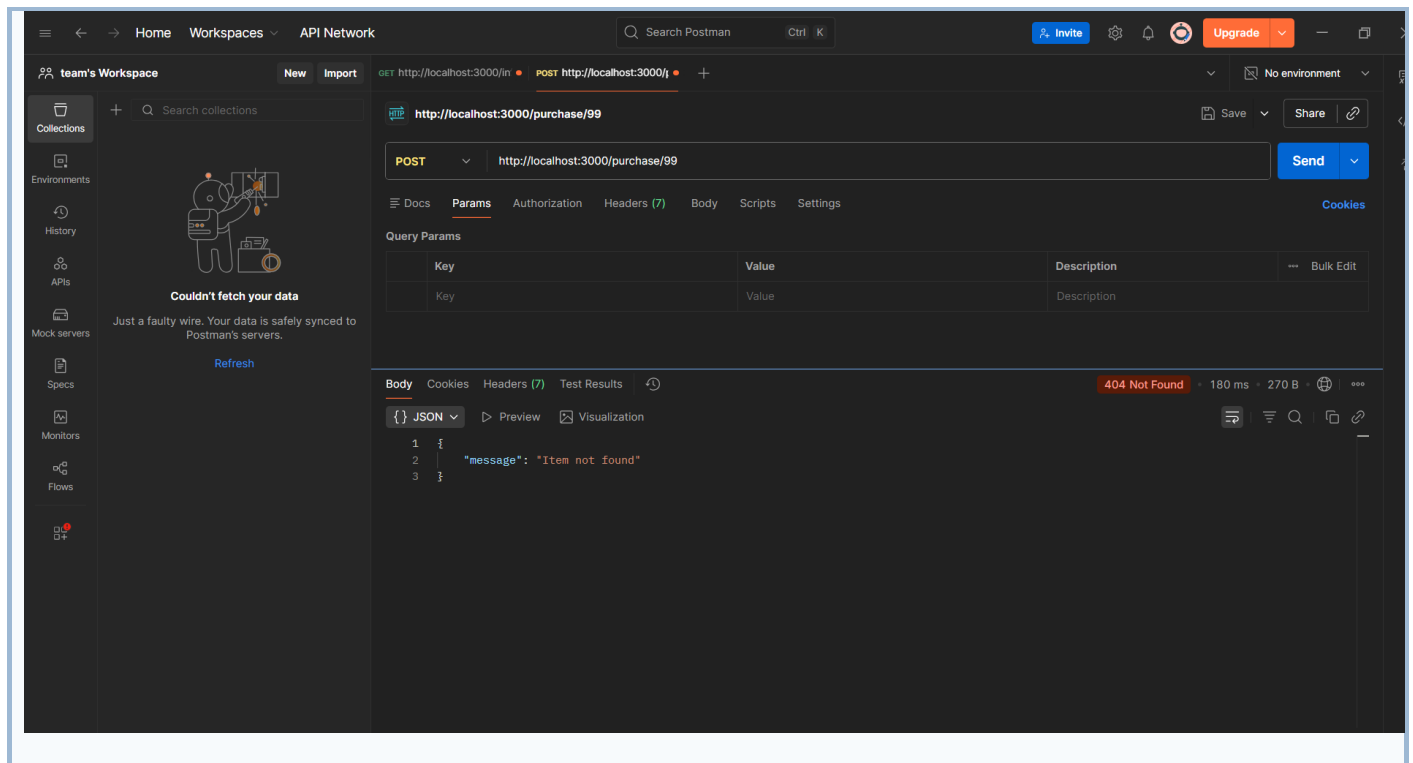


Figure 17. Invalid purchase request showing Item not found.

10

How to Run the Project

Commands used to build, run, and test the application.

The project can be run using Docker Compose. The database file should exist in the data directory, or it can be initialized using `init-db.js` before starting the containers.

Run commands

```
# Install dependencies locally if needed
npm install

# Initialize the SQLite database
node init-db.js

# Build and start all services
docker compose up --build

# Stop all services
docker compose down
```

Useful test URLs

```
GET http://localhost:3000/search/distributed systems
GET http://localhost:3000/search/undergraduate school
GET http://localhost:3000/info/2
POST http://localhost:3000/purchase/2
GET http://localhost:3001/query/item/2
POST http://localhost:3001/update/stock/2
POST http://localhost:3001/update/price/2
```

12

Design Tradeoffs and Improvements

Implementation decisions and possible extensions.

Design choice

The system uses SQLite instead of a heavyweight database. This keeps the project simple, portable, and suitable for a lab-scale microservices experiment.

- The frontend is simple and does not include a graphical interface because the project accepts browser/Postman or command-line clients.
- The services communicate using REST calls, which makes each service independently testable.
- SQLite is shared through a Docker volume so data persists after containers are restarted.
- A future version can add authentication, better input validation, a web user interface, and more detailed logging.
- Another future improvement is to add caching and replicas, which is implemented later in Lab 2.

Lab 1 successfully implements Bazar.com as a multi-tier microservices application. The frontend, catalog, and order services run separately and communicate using HTTP REST APIs. The catalog service stores and updates book information, while the order service verifies stock, decreases quantity, and records successful purchases. SQLite provides persistent storage, and Docker Compose makes the system easy to build and run.

Final result

The implementation satisfies the core Lab 1 requirements: search, info, purchase, persistent storage, REST communication, and containerized deployment.